

Crear Máquina Docker

Para crear un laboratorio o máquina vulnerable en Docker, existen dos métodos principales: uno interactivo (más sencillo si no dominas Docker) y otro mediante un archivo de configuración (Dockerfile).

Requisitos Previos

- Tener instalado **Docker** en tu sistema Linux (Kali Linux, Ubuntu, etc.).
- Conexión a internet para descargar las imágenes base.

Pasos de Instalación Recomendados

1. Actualizar el sistema e instalar dependencias:

```
sudo apt update
```

Instalar Docker (desde los repositorios):

```
sudo apt install -y docker.io
```

Aunque algunos prefieren el método del repositorio oficial de Docker por ser más reciente, docker.io funciona perfectamente para estos laboratorios.

Habilitar e iniciar el servicio Docker:

```
sudo systemctl enable docker --now
```

(--now ← Doble guión)

Recordatorio Importante (Post-instalación)

Si no quieres usar sudo antes de cada comando docker (lo cual es muy recomendable en Kali para la comodidad de uso), debes **añadir tu usuario al grupo docker** y luego reiniciar tu sesión o el terminal:

```
sudo usermod -aG docker $USER
```

Método 1: Creación Interactiva (Paso a Paso)

Este método consiste en configurar la máquina manualmente como si fuera una máquina virtual común.

1. Descargar la imagen base:

Utiliza una distribución ligera como Ubuntu.

```
sudo docker pull ubuntu:latest
```

2. Lanzar el contenedor e interactuar con él:

```
docker run -it <ID_DE_LA_IMAGEN>
```

```
sudo docker run -it bbdabce66f1b
```

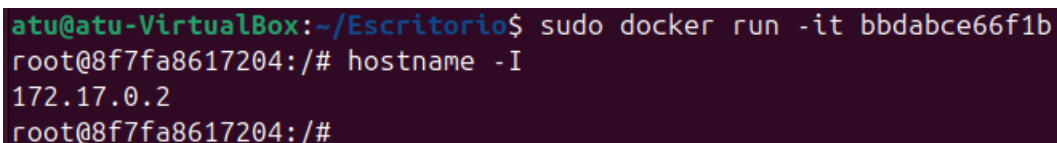
Para encontrar el ID de la imagen que necesitas, solo tienes que pedirle a Docker que te muestre qué es lo que tiene guardado en su "biblioteca" local.

```
sudo docker image ls
```

3. Identificar la IP del contenedor (para realizar pruebas desde fuera):

Dentro del contenedor, ejecuta:

```
hostname -I
```



```
atu@atu-VirtualBox:~/Escritorio$ sudo docker run -it bbdabce66f1b
root@8f7fa8617204:/# hostname -I
172.17.0.2
root@8f7fa8617204:/#
```

4. Instalar y configurar servicios:

Actualiza los repositorios e instala lo que necesites (Apache, SSH, FTP, etc.):

```
apt update
```

```
apt install apache2 openssh-server vsftpd -y
```

```
Please select the geographic area in which you live. Subsequent configuration
questions will narrow this down by presenting a list of cities, representing
the time zones in which they are located.

 1. Africa      4. Arctic    7. Australia 10. Pacific
 2. America    5. Asia      8. Europe    11. Etc
 3. Antarctica 6. Atlantic  9. Indian    12. Legacy
Geographic area: 8

Please select the city or region corresponding to your time zone.

 1. Amsterdam 14. Copenhagen 27. London    40. Riga      53. Ulyanovsk
 2. Andorra   15. Dublin     28. Luxembourg 41. Rome     54. Vaduz
 3. Astrakhan 16. Gibraltar 29. Madrid    42. Samara   55. Vatican
 4. Athens    17. Guernsey   30. Malta     43. San_Marino 56. Vienna
 5. Belfast   18. Helsinki  31. Mariehamn 44. Sarajevo  57. Vilnius
 6. Belgrade  19. Isle_of_Man 32. Minsk    45. Saratov   58. Volgograd
 7. Berlin    20. Istanbul  33. Monaco    46. Simferopol 59. Warsaw
 8. Bratislava 21. Jersey    34. Moscow    47. Skopje    60. Zagreb
 9. Brussels  22. Kaliningrad 35. Nicosia   48. Sofia     61. Zurich
10. Bucharest 23. Kirov     36. Oslo      49. Stockholm
11. Budapest  24. Kyiv      37. Paris     50. Tallinn
12. Busingen  25. Lisbon    38. Podgorica 51. Tirane
13. Chisinau  26. Ljubljana 39. Prague    52. Tiraspol

Time zone: 29

Current default time zone: 'Europe/Madrid'
```

5. Iniciar servicios y crear usuarios:

```
service apache2 start
```

```
service ssh start
```

```
adduser jose
```

```

root@8f7fa8617204:/# service ssh start
* Starting OpenBSD Secure Shell server sshd [ OK ]
root@8f7fa8617204:/# adduser jose
info: Adding user `jose' ...
info: Selecting UID/GID from range 1000 to 59999 ...
info: Adding new group `jose' (1001) ...
info: Adding new user `jose' (1001) with group `jose (1001)' ...
info: Creating home directory `/home/jose' ...
info: Copying files from `/etc/skel' ...
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for jose
Enter the new value, or press ENTER for the default
    Full Name []:
    Room Number []:
    Work Phone []:
    Home Phone []:
    Other []:
Is the information correct? [Y/n] y
info: Adding new user `jose' to supplemental / extra groups `users' ...
info: Adding user `jose' to group `users' ...
root@8f7fa8617204:/#

```

6. Guardar el progreso (Snapshots):

Desde una terminal **fuera** del contenedor, guarda el estado actual para no perder los cambios:

```

docker commit <ID_DEL_CONTENEDOR> nombre_imagen:latest
sudo docker commit 8f7fa8617204 nombre_imagen:latest

```

¿Cómo encontrar el ID correcto?

Ejecuta este comando para ver todos tus contenedores (los que están activos y los que se detuvieron):

```
sudo docker ps -a
```

```

atu@atu-VirtualBox:~/Escritorio$ sudo docker ps -a

```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
8f7fa8617204	bdbabce66f1b	"/bin/bash"	32 minutes ago	Up 32 minutes		epic_lumiere

```

atu@atu-VirtualBox:~/Escritorio$

```

7. Automatizar el arranque de servicios:

Prepara tu carpeta de trabajo

Es mejor no soltar archivos sueltos en el escritorio. Vamos a crear una carpeta específica:

1. Abre la terminal en tu escritorio.
2. Crea una carpeta llamada mi-proyecto (o el nombre que prefieras):

```
mkdir ~/Escritorio/docker-web
```

```
cd ~/Escritorio/docker-web
```

Como los servicios se detienen al exportar, debes crear un archivo llamado Dockerfile en tu carpeta de trabajo con este contenido:

nano Dockerfile

Dockerfile

```
FROM nombre_imagen:latest
```

```
CMD service ssh start && service apache2 start && service vsftpd  
start && tail -f /dev/null
```

Nota: El comando tail -f /dev/null es fundamental para que el contenedor no se apague solo.

8. Construir y Exportar la máquina final:

Construir la imagen final

```
sudo docker build -t mi_maquina .
```

Ojo: No olvides el **punto (.)** al final; le indica a Docker que busque el Dockerfile en la carpeta actual.

Exportar a formato .tar

```
docker save -o mi_maquina.tar mi_maquina:latest
```

Convertir tu trabajo actual en una Imagen

Como ya has hecho cambios dentro del contenedor (usuario "jose", servicios, etc.), primero guarda ese estado. En una terminal nueva, escribe:

```
sudo docker commit 8f7fa8617204 mi_maquina_base
```

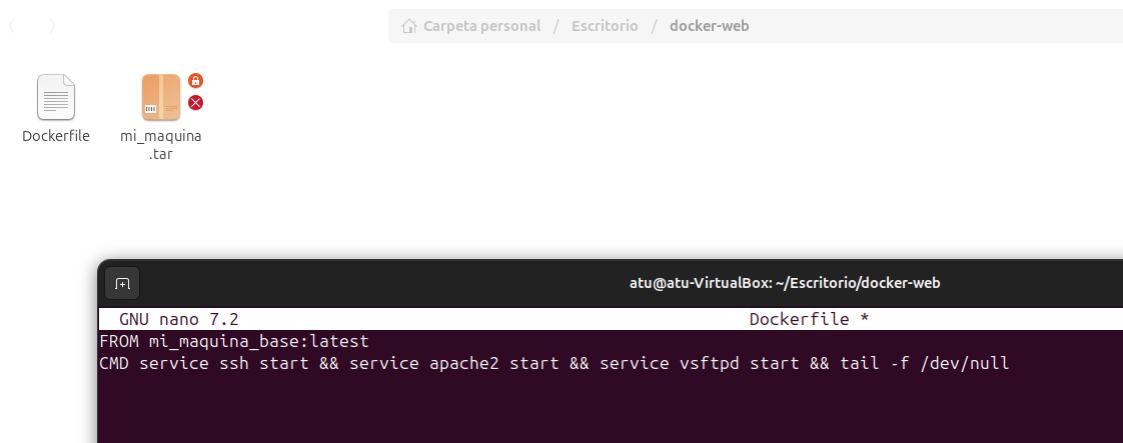
Ajustar el Dockerfile

Ahora, abre el archivo **Dockerfile** que tienes en la carpeta docker-web y asegúrate de que la primera línea coincida con el nombre que acabamos de crear:

Dockerfile

```
FROM mi_maquina_base:latest
```

```
CMD service ssh start && service apache2 start && service vsftpd start && tail -f /dev/null
```



Construir la imagen final ("Cocinarla")

Ahora que el Dockerfile sabe de dónde sacar la información, vamos a crear la imagen que arranca sola. Desde la terminal, dentro de ~/Escritorio/docker-web:

```
sudo docker build -t mi_maquina_automatica .
```

[!IMPORTANTE] El punto al final es obligatorio. Esto creará una imagen llamada mi_maquina_automatica que Docker ya podrá reconocer.

```

atu@atu-VirtualBox:~/Escritorio/docker-web$ sudo docker build -t mi_maquina_automatica .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
             Install the buildx component to build images with BuildKit:
             https://docs.docker.com/go/buildx/

Sending build context to Docker daemon 347.7MB
Step 1/2 : FROM mi_maquina_base:latest
--> e86aab957765
Step 2/2 : CMD service ssh start && service apache2 start && service vsftpd start && tail -f /dev/null
--> Running in 04e8c05f5da8
--> Removed intermediate container 04e8c05f5da8
--> 41d53d974eef
Successfully built 41d53d974eef
Successfully tagged mi_maquina_automatica:latest
atu@atu-VirtualBox:~/Escritorio/docker-web$

```

Lanzar el contenedor definitivo

Ahora sí, ejecuta el comando para que tu máquina funcione con los puertos abiertos y los servicios activos:

```
sudo docker run -d -p 80:80 -p 2222:22 --name contenedor_final mi_maquina_automatica
```

¿Cómo compruebo que realmente está funcionando?

Ahora mismo tu "máquina" está viva. Vamos a verificarlo con estos dos pasos:

A. Mira el contenedor en la lista Ejecuta esto en tu terminal:

```
sudo docker ps
```

```

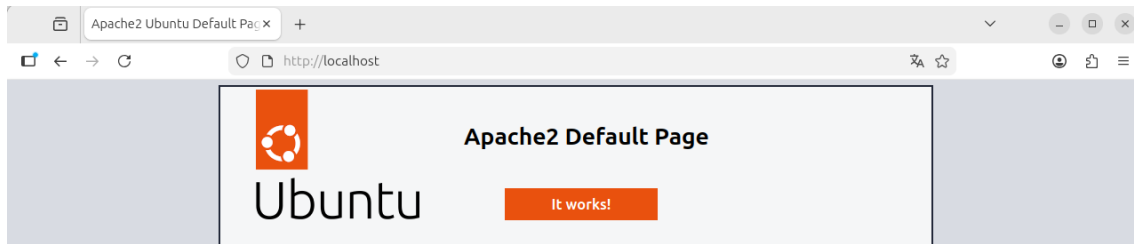
atu@atu-VirtualBox:~/Escritorio/docker-web$ sudo docker ps
CONTAINER ID   IMAGE                  COMMAND                  CREATED        STATUS        PORTS
7771ceb74f33   mi_maquina_automatica "/bin/sh -c 'service_..." About a minute ago Up About a minute   0.0.0.0:80->80/tcp, [::]:80->80/tcp, 0.0.0.0:2222->22/tcp, [::]:2222->22/tcp
contenedor_final

```

Deberías ver una línea con el nombre contenedor_final y los puertos 80 y 2222 mapeados.

B. Prueba los servicios Como configuramos el arranque automático, los servicios deberían estar respondiendo:

1. **Apache:** Abre el navegador en tu Ubuntu y entra en **http://localhost**. Deberías ver la página de bienvenida de Apache.
2. **SSH:** Intenta conectar con el usuario que creaste (jose) a través del puerto que redirigimos (2222):



`ssh jose@localhost -p 2222`

```
atu@atu-VirtualBox:~/Escritorio/docker-web$ ssh jose@localhost -p 2222
The authenticity of host '[localhost]:2222 ([127.0.0.1]:2222)' can't be established.
ED25519 key fingerprint is SHA256:1j+Zku3XaELVmiK5STcpCjqOd9gq9BDYYs0vkbtagck.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[localhost]:2222' (ED25519) to the list of known hosts.
jose@localhost's password:
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.14.0-37-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

jose@7771ceb74f33:~$
```

Un pequeño consejo sobre el espacio en disco

- **Sugerencia:** Una vez que compruebes que todo funciona, podrías mover el archivo `.tar` fuera de esa carpeta para que los futuros builds sean instantáneos.

Método 2: Usando un Dockerfile (Directo)

Si prefieres escribir toda la configuración en un solo archivo, crea un Dockerfile con una estructura similar a esta:

Dockerfile

```
FROM debian:latest
```

```
# Actualizar e instalar servicios y sudo
```

```
RUN apt update && apt upgrade -y && apt install apache2 vsftpd  
openssh-server sudo -y
```

```
# Crear usuarios y configurar contraseñas
```

```
RUN useradd -m bob && echo "atu:dingdong" | chpasswd
```

```
RUN useradd -m mario && echo "jose:dingdong" | chpasswd
```

```
# Configurar permisos SUID o SUDO para escalada de privilegios  
(opcional)
```

```
RUN chmod u+s /usr/bin/find
```

```
# Comando para iniciar servicios y mantener vivo el contenedor
```

```
CMD service apache2 start && service vsftpd start && service ssh  
start && tail -f /dev/null
```

Para finalizar este método:

1. Construye la imagen:

```
docker build -t mi_maquina_pro .
```

2. Exporta la máquina:

```
docker save -o mi_maquina_pro.tar mi_maquina_pro:latest
```

Una vez tengas el archivo .tar, es el que se utiliza para compartir o subir a plataformas como **DockerLabs**.